

SUDOKU PUZZLE SOLVER

UCS3212 - Fundamentals and Practice of Software Development

A PROJECT REPORT

Submitted by

1. Sai Keerthi Reddy - 3122255001156
2. Sophia Florence - 3122255001182
3. Varshath K - 3122255001202



Department of Computer Science and Engineering

Sri Sivasubramaniya Nadar College of Engineering
(An Autonomous Institution, Affiliated to Anna University)
Kalavakkam – 603110

May 2026

Sri Sivasubramaniya Nadar College of Engineering
(An Autonomous Institution, Affiliated to Anna University)

BONAFIDE CERTIFICATE

Certified that this project report titled “SUDOKU PUZZLE SOLVER” is the bonafide work of Sai Keerthi Reddy (3122255001156), Sophia Florence (3122255001182) and Varshath K (3122255001202) who carried out the project work in the UCS3212 - Fundamentals and Practice of Software Development during the academic year 2025-26.

Internal Examiner

External Examiner

Date:

Contents

1	Abstract	4
2	Introduction	5
3	Problem Statement	6
4	Exploration of Problem Statement	7
4.1	Key Functionalities	7
4.2	Challenges & Constraints	7
5	Limitations of Existing Systems	8
5.1	Focus on Scheduling Constraints	8
5.2	Future Expansion	8
6	Architecture Diagram with Explanation	9
7	Function Flowcharts	11
7.1	Main Entry Point	11
7.2	Puzzle Generation Module (solver.c)	12
7.3	Game Play Module (game.c)	15
7.4	Leaderboard Module	17
7.5	Display / UI Utilities Module (utils.c)	18
8	Modules with Constraints	19
8.1	Puzzle Generation Module	19
8.2	Game Play Module	19
8.3	Auto Solver Module	20
8.4	Leaderboard Module	21
8.5	Display / UI Module	21
9	Implementation	23
9.1	Data Organisation	23
9.2	Development Tools	23
9.3	Libraries Used	23
9.4	User Interface Design	23
9.5	Code Implementation	24
10	Observations with Respect to Society	26
11	Legal and Ethical Perspectives	27
12	Limitations and Future Enhancements	28
13	Learning Outcomes	29

1. Abstract

Sudoku Puzzle Solver is a console-based application written in C that offers two core experiences: an interactive puzzle-play mode and a fully automatic solver. The play mode lets a user select a difficulty level, enter a name, and solve a freshly generated 9x9 Sudoku puzzle with a live timer, colour-coded feedback, and a regenerating hint system. The auto-solver mode accepts either a system-generated or a user-inputted puzzle and solves it using a backtracking algorithm enhanced with the Minimum Remaining Values (MRV) heuristic and Forward Checking, displaying a step-by-step explanation of every placement decision. Scores are persisted to a leaderboard file and displayed as a ranked table. The project is implemented across five C source files (main.c, globals.c, utils.c, solver.c, game.c) and a shared header (sudoku.h), using only standard C libraries and the Windows Console API for colour and cursor control.

2. Introduction

Sudoku is a globally recognised logic puzzle in which the digits 1 through 9 must be placed in a 9x9 grid so that every row, every column, and every 3x3 sub-grid contains each digit exactly once. Despite its simple rules, the puzzle presents rich computational challenges that make it an ideal subject for exploring constraint-satisfaction algorithms and user-interface design in a systems-programming language.

Sudoku Puzzle Solver extends a first-version prototype with several new capabilities: a randomised puzzle generator using diagonal-fill seeding, a difficulty selector that controls the number of pre-filled cells, a regenerating hint system, an automated step-by-step solver visualiser, and a persistent leaderboard. The application targets Windows and is compiled with GCC; it makes no use of external graphics libraries, relying instead on the Windows Console API for terminal-width detection, cursor positioning, and text colouring.

3. Problem Statement

A standard Sudoku puzzle consists of a 9×9 grid, divided into nine 3×3 sub-grids. Some cells are pre-filled with digits from 1 to 9, and the remaining cells are empty.

The goal is to fill the empty cells such that:

- Each row contains the digits 1 to 9 exactly once •
- Each column contains the digits 1 to 9 exactly once •
- Each 3×3 sub-grid contains the digits 1 to 9 exactly once

Difficulty levels

1. Easy level (Mandatory)

- a. Provide more pre-filled numbers.
- b. Ensure minimal recursion
- c. Mostly single-candidate placements

2. Medium level (Mandatory)

- a. Moderate number of blanks
- b. Requires elimination logic
- c. Moderate recursion depth

3. Hard level (Optional)

- a. Fewer initial clues
- b. Requires deeper recursion
- c. May require advanced pruning techniques

4. Exploration of Problem Statement

4.1 Key Functionalities

- **Puzzle Generation:** The diagonal 3x3 boxes are filled first with a shuffled 1-9 array (always valid, no constraint checks needed). The remaining cells are filled by the backtracking solver. Holes are then dug randomly based on difficulty.
- **Interactive Play:** Arrow-key navigation, numeric entry, zero/backspace deletion, hint request (H key), and ESC exit are handled in a polling loop that also redraws the timer every second.
- **Auto Solver – Automated Mode:** A puzzle is generated internally; the user presses Solve to trigger the backtracking solver and view a side-by-side original-vs-solved display.
- **Auto Solver – User-Input Mode:** The user enters up to a configurable maximum of their own clue digits; validation checks for pre-existing row/column/box conflicts before solving begins.
- **Step-by-Step Solver:** After solving, the MRV order is replayed step by step with explanations identifying which row, column, and box constraints eliminated which candidate digits.
- **Leaderboard:** Scores are written to leaderboard.txt in append mode, read back, sorted by ascending time, and the top-ten entries are displayed.

4.2 Challenges & Constraints

- **Sudoku Validity:** A generated puzzle must have at least one solution. The hole-digging step uses a simple random approach and does not guarantee a unique solution; uniqueness is an acknowledged limitation.
- **Solver Efficiency:** Naive backtracking has worst-case exponential complexity. MRV and Forward Checking are applied to prune the search tree significantly.
- **Terminal Rendering:** Cursor positioning and colour require Windows Console API calls. The grid is redrawn in-place (no flicker) using gotoxy() rather than full screen clears during play.
- **Hint Regeneration Timing:** The system tracks a last-regen timestamp and adds one hint credit per 20 seconds elapsed, capped at 3.

5. Limitations of Existing Systems

5.1 Focus on Scheduling Constraints

Most open-source console Sudoku implementations offer only a static puzzle with no difficulty selection, no live timer, and no solver explanation. This project addresses those gaps by combining puzzle generation, interactive play, and educational step-by-step solving in a single application.

5.2 Future Expansion

- Cross-platform support by replacing Windows Console API calls with a portable library such as ncurses.
- Guaranteed unique-solution puzzle generation via a solve-count check after each hole is dug.
- AI-difficulty adaptation that adjusts hole count based on the player's historical solve times.

6. Architecture Diagram with Explanation

The application is split into five compilation units that communicate through shared global variables declared in `globals.c` and prototyped in `sudoku.h`.

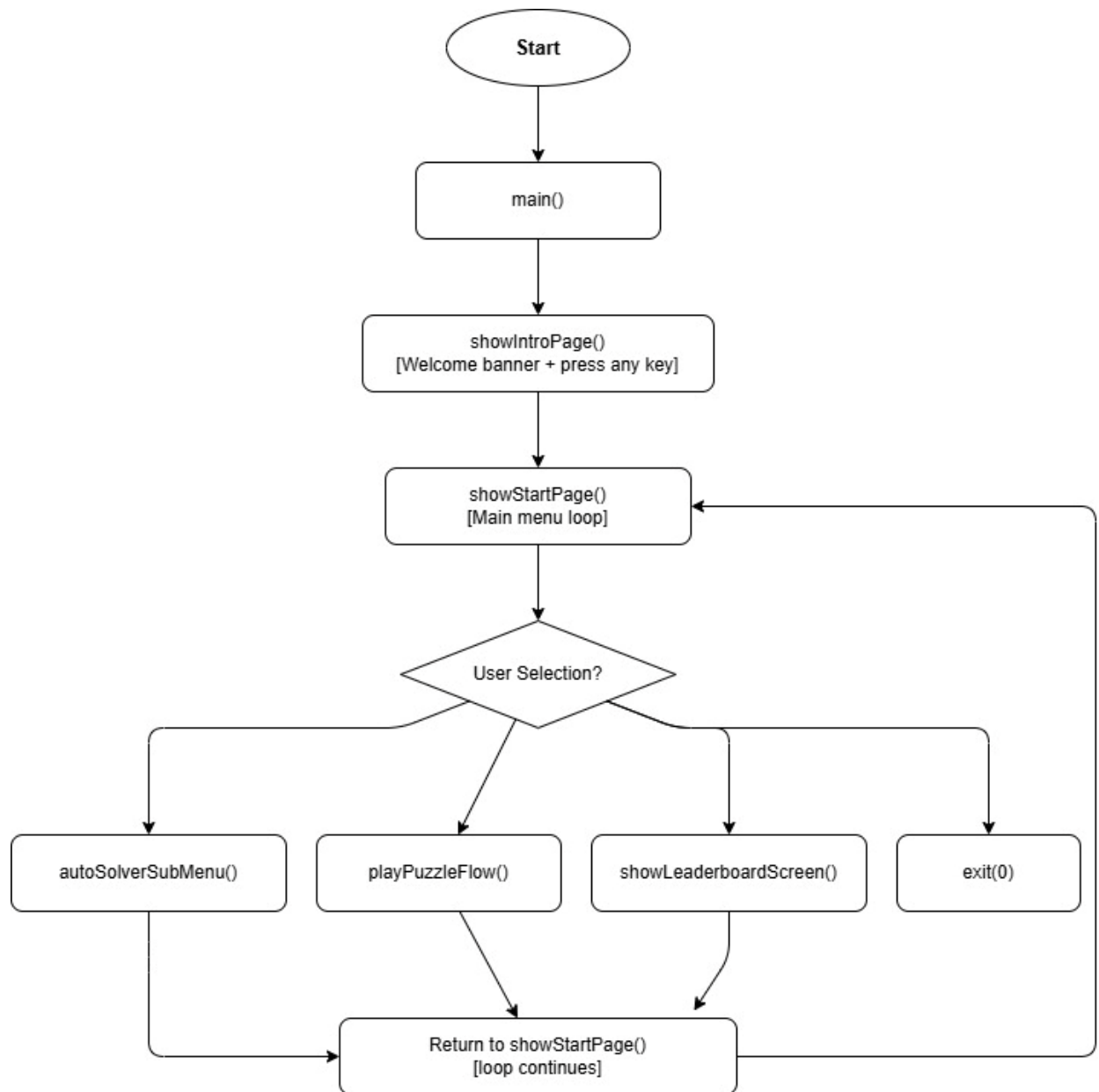


Figure 6: Overall Architecture Diagram

Key architectural components:

- `globals.c` – Defines `grid[9][9]`, `original_grid[9][9]`, `solution_grid[9][9]`, `playerName`, `difficultyHoles`, and `solver_complexity`. All modules share these via extern declarations in `sudoku.h`.
- `utils.c` – Console utility layer: terminal-size detection (`getTerminalWidth / getTerminalHeight`), centred printing, colour setting (`setColor`), cursor positioning (`gotoxy`), and cursor visibility control.
- `solver.c` – Puzzle logic: `isSafe()`, `countCandidates()`, `forwardCheck()`, `solveGrid()` (MRV + FC backtracking), `shuffleArray()`, `fillDiagonal()`, `generatePuzzle()`.
- `game.c` – All UI flows: intro page, start page (menu), play puzzle, auto solver (automated and user-input), step-by-step viewer, win handling, and leaderboard display.
- `sudoku.h` – Single shared header: includes, extern globals, the `LeaderboardEntry` typedef, and all function prototypes.

7. Function Flowcharts

7.1 Main Entry Point

main() loops forever calling showIntroPage() then showStartPage(). showStartPage() presents a four-option menu and dispatches to the appropriate flow.

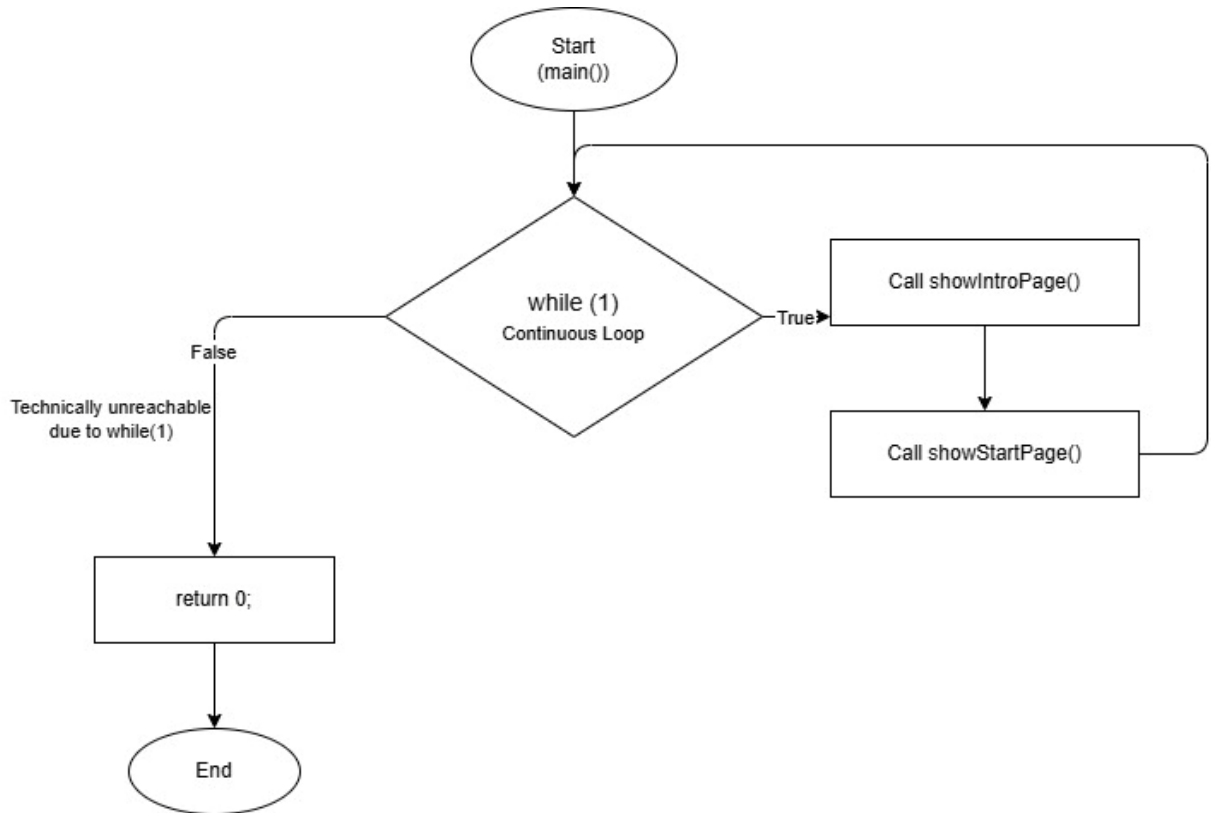


Figure 7.1: Main Module Architecture Diagram

7.2 Puzzle Generation Module (solver.c)

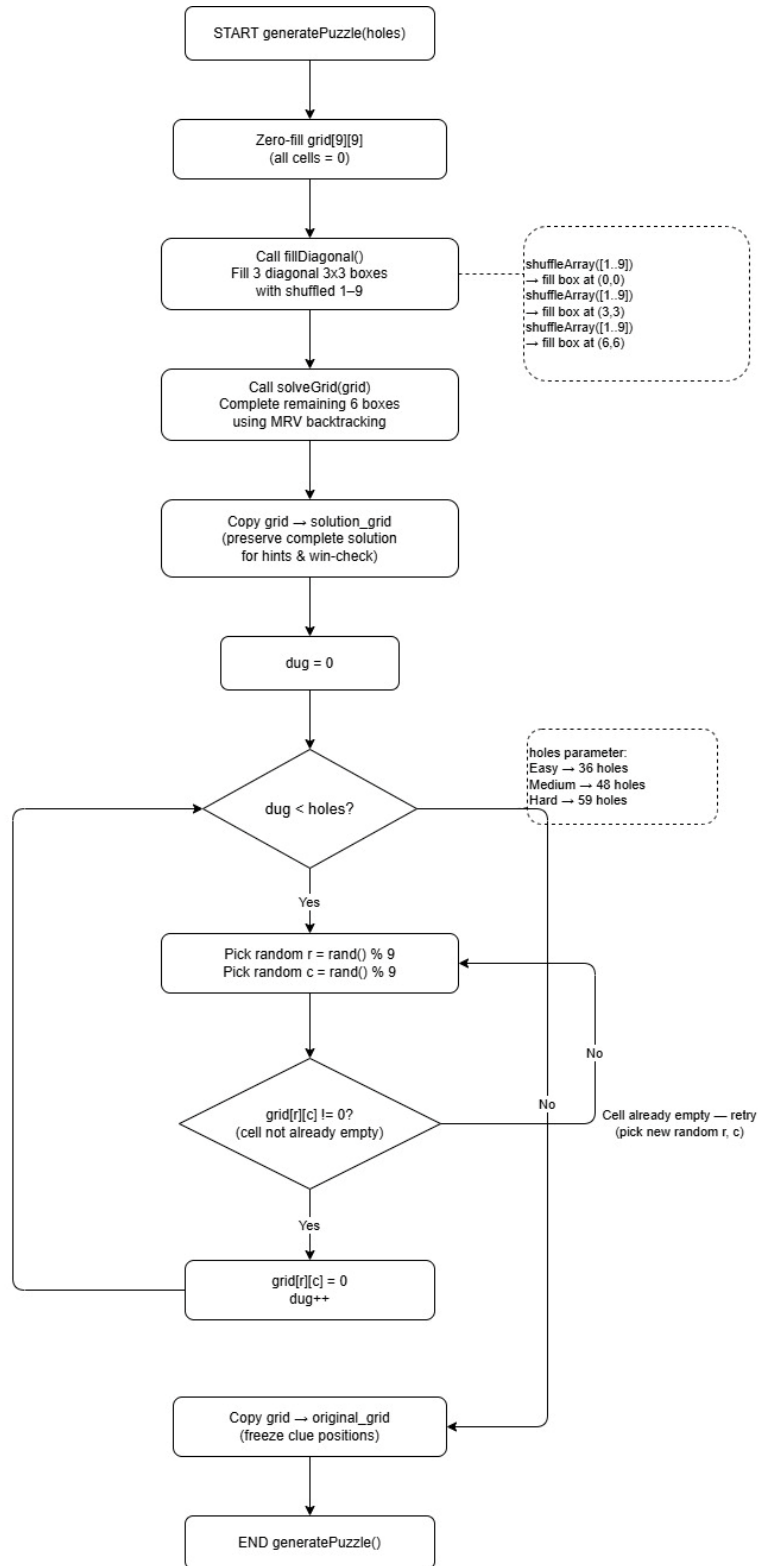


Figure 7.2.1: Puzzle Generation Module Architecture Diagram

Auto solver(MRV + Forward Checking backtracker):

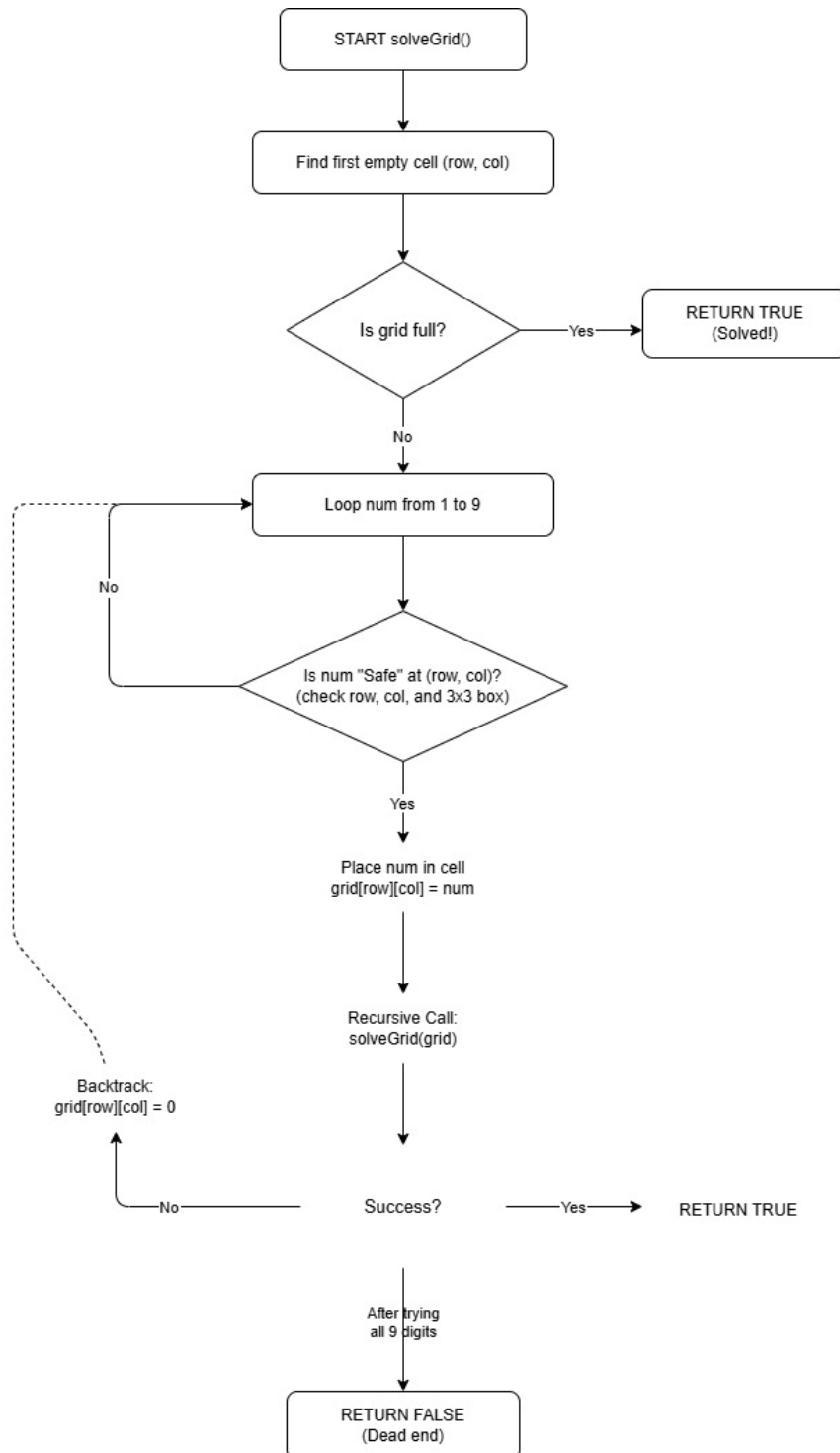


Figure 7.2.2: Solver Module Architecture Diagram

```

=== EFFICIENT SOLVING STEPS ===

Original Puzzle      Solving Progress
+-----+-----+ +-----+-----+
| . 8 2 | . . 7 | 9 . 4 | | 3 8 2 | 1 5 7 | 9 6 4 |
| . . . | 6 3 . | . 2 . | | 1 4 7 | 6 3 9 | . 2 . |
| 5 6 9 | 2 4 . | . . . | | 5 6 9 | 2 4 8 | . 1 . |
+-----+-----+ +-----+-----+
| . . 8 | 5 . . | . 7 . | | . . 8 | 5 . . | . 7 . |
| 6 . 3 | . 8 . | 2 . . | | 6 . 3 | 4 8 . | 2 . . |
| . . 4 | 7 . 6 | . 3 1 | | . . 4 | 7 . 6 | . 3 1 |
+-----+-----+ +-----+-----+
| 4 . . | 9 . 5 | . 8 . | | 4 . . | 9 . 5 | . 8 . |
| . 3 . | 8 . . | 1 4 . | | . 3 . | 8 . . | 1 4 . |
| . . 1 | 3 . . | . . 2 | | . . 1 | 3 . . | . . 2 |
+-----+-----+ +-----+-----+

Step 11 of 45 | Complexity: 46 backtrack calls

-> Placed 4 at Row 5, Col 4 (ONLY valid option!)
    Only 4 can go here. Row 5 eliminates {2,3,6,8}, Col 4 {1,5,7,9}, Box 5 {} {}
    Valid candidates: {4}

[Left] / [Right] Prev/Next Step    [ESC] Back

```

Figure 7.2.3: Auto-solver Using Backtracking Algorithm

7.3 Game Play Module (game.c)

playPuzzleFlow() collects player name, difficulty selection, generates the puzzle, then enters the play loop.

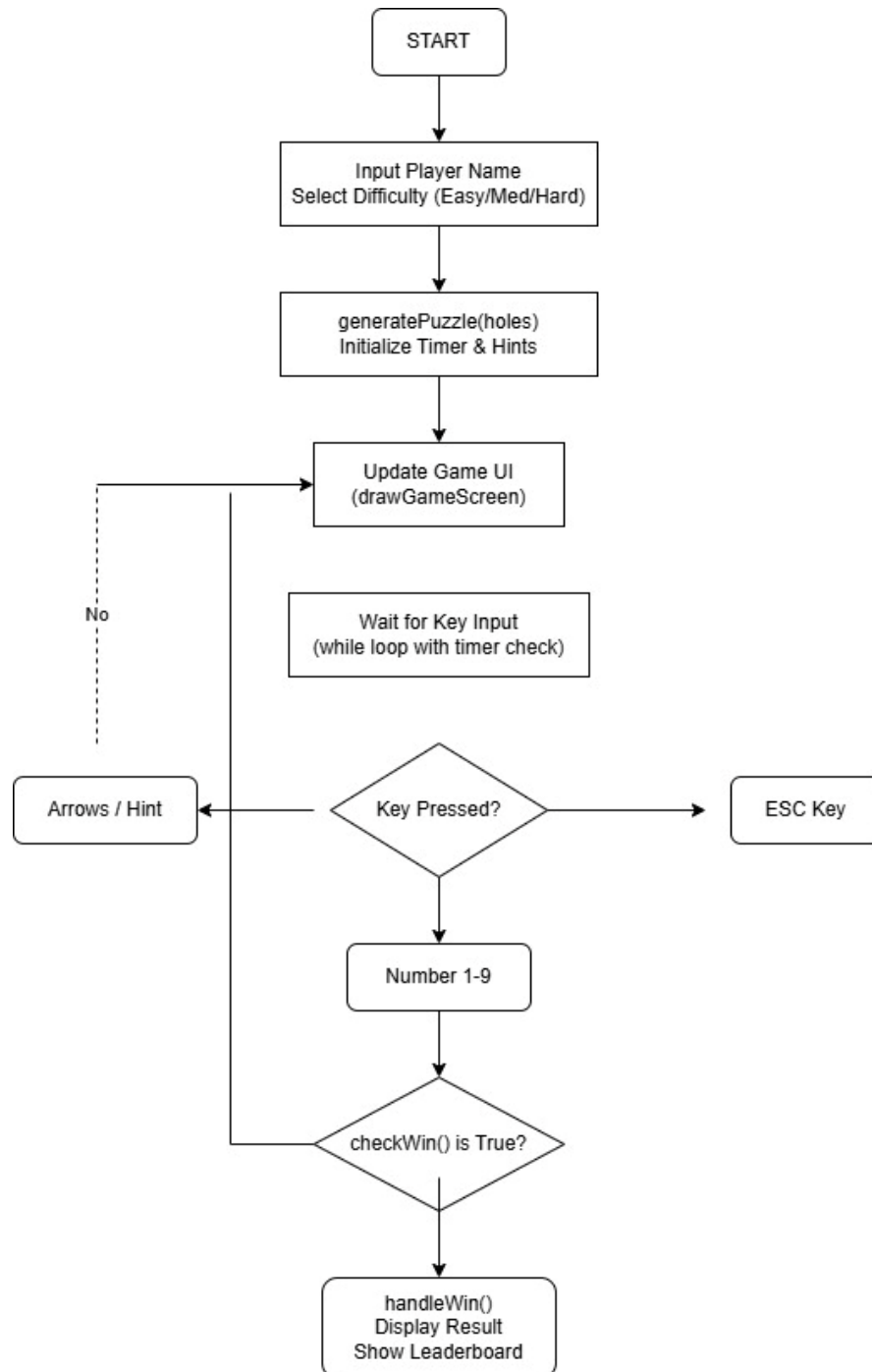


Figure 7.3.1: Game Module Architecture Diagram

stepByStepSolverUI() – step-by-step viewer:

```
=== EFFICIENT SOLVING STEPS ===

Original Puzzle      Solving Progress
+-----+-----+-----+ +-----+-----+-----+
| 2  .  . | .  4  . | .  .  . | | 2  1  6 | 7  4  8 | 9  3  5 |
| .  .  . | .  .  . | .  .  1 | | 7  4  9 | 3  2  5 | 6  8  1 |
| .  .  . | .  .  . | .  .  . | | 3  5  8 | 1  6  9 | 2  4  7 |
+-----+-----+-----+ +-----+-----+-----+
| .  .  . | .  .  . | .  5  . | | 9  3  1 | .  7  2 | 4  5  . |
| 6  .  . | .  5  . | .  .  . | | 6  8  4 | 9  5  3 | .  1  . |
| .  .  . | .  .  . | .  .  . | | 5  2  7 | .  .  . | .  9  . |
+-----+-----+-----+ +-----+-----+-----+
| .  .  . | .  .  . | .  .  . | | .  9  . | .  .  . | .  6  . |
| .  .  . | .  .  . | .  .  . | | .  6  . | .  .  . | .  7  . |
| .  7  . | .  .  . | .  .  9 | | .  7  . | .  1  . | .  2  9 |
+-----+-----+-----+ +-----+-----+-----+

Step 45 of 73 | Complexity: 77 backtrack calls

-> Placed 3 at Row 5, Col 6 (ONLY valid option!)
    Only 3 can go here. Row 5 eliminates {1,4,5,6,8,9}, Col 6 {2}, Box 5 {7}5 {}
    Valid candidates: {3}

[Left] / [Right] Prev/Next Step    [ESC] Back
```

Figure 7.3.2: Step By Step Solver Output

7.4 Leaderboard Module

Scores are saved on win and displayed from leaderboard.txt.

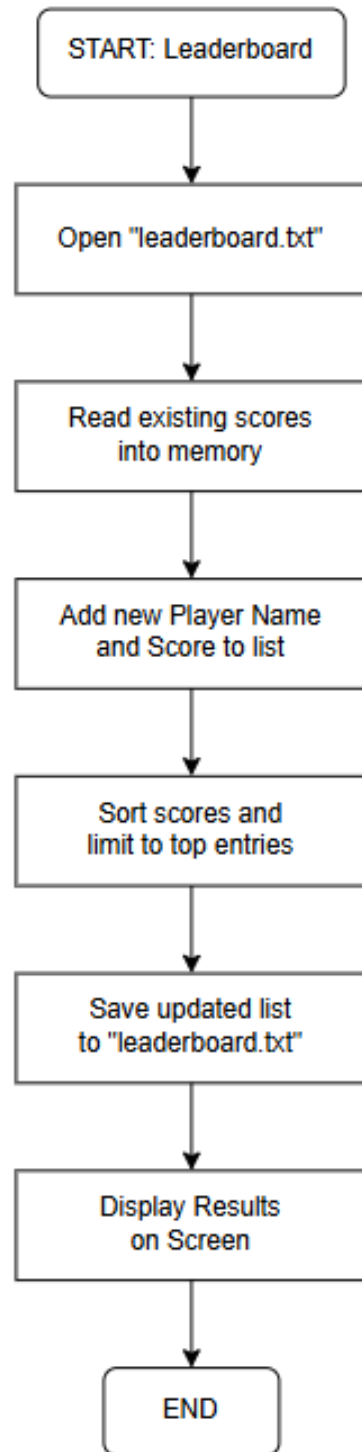


Figure 7.4: Leaderboard Module Architecture Diagram

7.5 Display / UI Utilities Module (utils.c)

utils.c provides terminal-measurement and rendering primitives used throughout game.c.

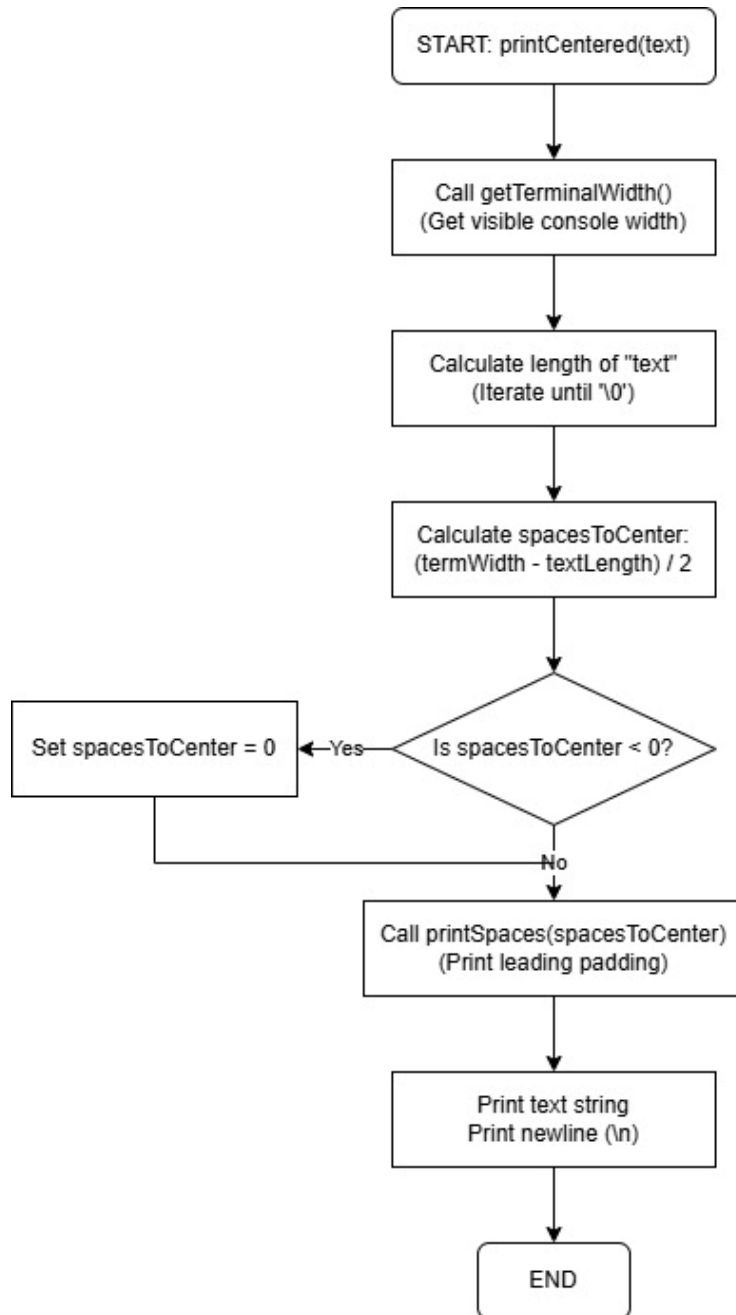


Figure 7.5: Utilities Module Architecture Diagram

8. Modules with Constraints

8.1 Puzzle Generation Module

Description:

Implemented in solver.c. Generates a valid Sudoku puzzle by seeding the three diagonal 3x3 boxes, completing the grid via backtracking, storing the full solution, and then removing cells (digging holes) to create the puzzle.

Constraints:

- Grid is always 9x9; exactly 81 cells.
- Difficulty controls the number of holes: Easy = 36, Medium = 48, Hard = 59.
- The hole-digging step is random; unique-solution guarantee is not enforced.
- solution_grid is populated before holes are dug so hints are always correct.

Functionalities:

- fillDiagonal(): fills the three independent diagonal boxes with a shuffled 1-9 array.
- solveGrid(): MRV + Forward-Checking backtracker; returns true if solved, false if unsolvable.
- generatePuzzle(holes): orchestrates generation, solution storage, and hole digging.

8.2 Game Play Module

Description:

Implemented in game.c (playPuzzleFlow and drawGameScreen). Manages the interactive puzzle session including input handling, live rendering, hint management, and win detection.

Constraints:

- Pre-filled cells (original_grid != 0) cannot be overwritten or cleared.
- Hint credits are capped at 3 and regenerate one credit per 20 seconds.
- The timer redraws at most once per second to avoid unnecessary CPU usage.
- Color codes: grey = original cell, green = valid user entry, red = invalid user entry, bracketed cursor.

Functionalities:

- drawGameScreen(): renders the centred grid in-place with colour-coded values and live timer.
- playPuzzleFlow(): collects name, handles difficulty selection, runs the game loop.
- checkWin(): returns true when every cell matches solution_grid with no empty cells remaining.
- handleWin(): records the time, appends to leaderboard, and triggers the leaderboard view.



Figure 8.2: Output From the Gameplay Module

8.3 Auto Solver Module

Description:

Implemented in game.c (autoSolverSubMenu, automatedPuzzleFlow, autoSolverFlow, stepByStepSolverUI, showSideBySideSolutionFlow). Provides automated and user-driven puzzle solving with optional step-by-step explanation.

Constraints:

- User-input mode limits clue count by difficulty: Easy ≤ 36 , Medium ≤ 27 , Hard ≤ 20 .
- Input validation rejects grids with pre-existing row/column/box conflicts.
- At least one clue must be entered before the solver is invoked.
- solver_complexity is reset to 0 before each solve call.

Functionalities:

- automatedPuzzleFlow(): generates a puzzle internally and offers Solve or Back.
- autoSolverFlow(): interactive grid input with conflict detection, then invokes solveGrid().
- showSideBySideSolutionFlow(): shows original and solved grids side by side; offers step-by-step view.
- stepByStepSolverUI(): replays the MRV solving order with per-step constraint explanations.

```

=== SOLUTION FOUND ===

Original Puzzle
+-----+-----+-----+
| . 8 2 | . . 7 | 9 . 4 |
| . . . | 6 3 . | . 2 . |
| 5 6 9 | 2 4 . | . . . |
+-----+-----+-----+
| . . 8 | 5 . . | . 7 . |
| 6 . 3 | . 8 . | 2 . . |
| . . 4 | 7 . 6 | . 3 1 |
+-----+-----+-----+
| 4 . . | 9 . 5 | . 8 . |
| . 3 . | 8 . . | 1 4 . |
| . . 1 | 3 . . | . . 2 |
+-----+-----+-----+

Solved Puzzle
+-----+-----+-----+
| 3 8 2 | 1 5 7 | 9 6 4 |
| 1 4 7 | 6 3 9 | 5 2 8 |
| 5 6 9 | 2 4 8 | 3 1 7 |
+-----+-----+-----+
| 2 1 8 | 5 9 3 | 4 7 6 |
| 6 7 3 | 4 8 1 | 2 9 5 |
| 9 5 4 | 7 2 6 | 8 3 1 |
+-----+-----+-----+
| 4 2 6 | 9 1 5 | 7 8 3 |
| 7 3 5 | 8 6 2 | 1 4 9 |
| 8 9 1 | 3 7 4 | 6 5 2 |
+-----+-----+-----+

Complexity (Backtrack calls): 46

Do you want to see the solving steps?
1. Yes, show steps
-> 2. No, go back      <-

```

Figure 8.3: Output From the Auto-Solver Module

8.4 Leaderboard Module

Description:

File-based persistence using leaderboard.txt. Entries are name-time pairs appended on every win. On display, up to 100 entries are read, sorted, and the top-10 shown.

Constraints:

- leaderboard.txt is opened in append mode on write and read mode on display; missing file shows 'No scores recorded yet'.
- Maximum of 100 entries are loaded into memory at a time.
- Sorted ascending by timeSec using qsort with compareEntries comparator.
- Only the top 10 entries are displayed regardless of total count.

Functionalities:

- handleWin(): appends the current player name and elapsed seconds.
- showLeaderboardScreen(): reads, sorts, and displays the ranked leaderboard.

8.5 Display / UI Module

Description:

Console-based rendering using Windows Console API. All screens are centred based on real-time terminal width. Colour is applied per-character using ANSI colour codes via SetConsoleTextAttribute.

Constraints:

- The application is Windows-only due to reliance on <windows.h>, <conio.h>, and the Windows Console API.

- Terminal size is queried live; a fallback of 80x30 is used if the API fails.
- Colour codes are plain integers (e.g. 7 = white, 10 = green, 12 = red, 11 = cyan, 14 = yellow, 8 = grey).
- The cursor is hidden during rendering and restored on exit.

Functionalities:

- `getTerminalWidth()` / `getTerminalHeight()`: live terminal measurement.
- `getCenterMargin(contentWidth)`: returns left padding for centred display.
- `setColor(code)`: applies text colour attribute.
- `gotoxy(x,y)`: positions the cursor for in-place screen updates.
- `hideCursor()` / `showCursor()`: toggles cursor visibility.
- `printCentered(text)`: prints a line centred in the current terminal width.

9. Implementation

9.1 Data Organisation

- `grid[9][9]` – the live working grid modified by the player or solver.
- `original_grid[9][9]` – the initial puzzle state; cells `!= 0` are protected from editing.
- `solution_grid[9][9]` – the fully solved grid; used for hint lookup and win checking.
- `playerName[50]` – entered at the start of each puzzle session.
- `difficultyHoles` – number of cells removed from the complete grid (default 40).
- `solver_complexity` – counts recursive calls in `solveGrid()`; displayed as a complexity metric.
- `leaderboard.txt` – plain-text file; each line is "name seconds".

9.2 Development Tools

- Compiler: GCC (via Makefile with `-o sudoku` flag).
- IDE: Visual Studio Code with C/C++ extension (`c_cpp_properties.json` configured).
- Build: `make` or direct `gcc` invocation.
- Target OS: Windows (uses `windows.h`, `conio.h`).

9.3 Libraries Used

9.3.1 Standard C Libraries

- `stdio.h` – `printf`, `scanf`, `fopen`, `fclose`, `fscanf`, `fprintf`, `sprintf`, `sscanf`.
- `stdlib.h` – `exit`, `srand`, `rand`, `qsort`, `malloc` (implicitly via runtime).
- `string.h` – `strcmp`, `strcat`, `strlen`.
- `time.h` – `time()`, `difftime()` for elapsed timer and RNG seed.
- `stdbool.h` – `bool` type.

9.3.2 Windows-Specific Libraries

- `windows.h` – `GetConsoleScreenBufferInfo`, `SetConsoleTextAttribute`, `SetConsoleCursorPosition`, `SetConsoleCursorInfo`, `MessageBox`, `Sleep`.
- `conio.h` – `getch()`, `_kbhit()` for non-blocking keyboard input.

9.4 User Interface Design

Intro Page:

A centred ASCII banner reading 'WELCOME TO SUDOKU PUZZLE SOLVER' with a tagline and 'PRESS ANY KEY TO START' prompt.

Start Menu:

Four options (Auto Solver, Puzzle, Leaderboard, Exit) navigated with arrow keys. The selected option is highlighted with '`->`' and '`<-`' markers.

Game Screen:

A 9x9 grid drawn with pipe and plus characters, colour-coded digits, a bracketed cursor, a live MM:SS timer, player name, and a controls footer showing hint count.

Auto Solver Screen:

Side-by-side original and solved grids with colour differentiation: cyan for user-provided clues, green for solver-filled cells. The step-by-step view adds a step counter and per-step constraint explanation.

9.5 Code Implementation

9.5.1 Core Data Structures

```
// Leaderboard entry
typedef struct {
    char name[50];
    int timeSec;
} LeaderboardEntry;

// Step record for step-by-step solver
typedef struct {
    int r, c, val, numOptions;
    int candidates[9];    int candidateCount;
    int rowElim[9];       int rowElimCount;
    int colElim[9];       int colElimCount;
    int boxElim[9];       int boxElimCount;
    char reason[256];
} Step;
```

9.5.2 Key Algorithms

isSafe() – Constraint Validation

```
bool isSafe(int testGrid[9][9], int row, int col, int num) {
    for (int x = 0; x < 9; x++) {
        if (testGrid[row][x] == num || testGrid[x][col] == num)
            return false;
    }
    int startRow = row - row % 3, startCol = col - col % 3;
    for (int i = 0; i < 3; i++)
        for (int j = 0; j < 3; j++)
            if (testGrid[i+startRow][j+startCol] == num) return false;
    return true;
}
```

solveGrid() – MRV + Forward-Checking Backtracker

```
bool solveGrid(int testGrid[9][9]) {
    solver_complexity++;
    int bestRow = -1, bestCol = -1, minCandidates = 10;
    for (int i = 0; i < 9; i++) {
        for (int j = 0; j < 9; j++) {
            if (testGrid[i][j] == 0) {
                int c = countCandidates(testGrid, i, j);
                if (c == 0) return false;
                if (c < minCandidates) {
```

```

        minCandidates = c; bestRow = i; bestCol = j;
        if (minCandidates == 1) break;
    }
}
}
if (minCandidates == 1) break;
}
if (bestRow == -1) return true; // Solved
for (int num = 1; num <= 9; num++) {
    if (isSafe(testGrid, bestRow, bestCol, num)) {
        testGrid[bestRow][bestCol] = num;
        if (forwardCheck(testGrid, bestRow, bestCol))
            if (solveGrid(testGrid)) return true;
        testGrid[bestRow][bestCol] = 0;
    }
}
return false;
}

```

Hint Regeneration Logic

```

// Inside play loop (every key event and timer tick):
int elapsedRegenSecs = (int)diffTime(currentTime, lastHintRegenTime);
if (elapsedRegenSecs >= 20) {
    int regenAmount = elapsedRegenSecs / 20;
    hintsAvailable += regenAmount;
    if (hintsAvailable >= 3) {
        hintsAvailable = 3;
        lastHintRegenTime = currentTime;
    } else {
        lastHintRegenTime += regenAmount * 20;
    }
}
}

```

10. Observations with Respect to Society

10.1 Educational Value

Sudoku is widely used in cognitive training for people of all ages. The step-by-step solver explains constraint-satisfaction reasoning in plain English, making the application a teaching tool for logical thinking as well as an entertaining game.

10.2 Accessibility

The three difficulty levels lower the barrier for beginners while the hard mode and the auto-solver comparison challenge experienced players. The hint system provides on-demand assistance without revealing the full solution.

10.3 Algorithmic Literacy

Exposing `solver_complexity` (the count of recursive backtracking calls) alongside the solution teaches users that different puzzles have vastly different computational costs and introduces concepts such as search trees and heuristic pruning in an approachable context.

10.4 Future Outlook

- Multiplayer leaderboard via a shared server file or network socket.
- Accessibility features: high-contrast colour themes, larger font terminals.
- Mobile or web port using WebAssembly to reach a broader audience.

11. Legal and Ethical Perspectives

11.1 Open-Source Considerations

The application uses only the C standard library, Windows SDK headers (redistributed freely with any Windows installation), and no third-party components. There are no licensing obligations beyond acknowledging the standard library authors.

11.2 Data Privacy

The leaderboard stores only a player-supplied name and an integer elapsed time. No personal data, network communication, or telemetry is collected. The leaderboard file resides locally on the user's machine.

11.3 Fair Play

The hint system is rate-limited to prevent trivial completion. Using the Auto Solver is an explicitly separate menu option, so leaderboard scores reflect genuine manual play.

11.4 Algorithmic Transparency

The step-by-step solver view makes every placement decision fully transparent: the candidate list, constraint eliminations by row, column, and box, and the `solver_complexity` counter are all displayed to the user.

12. Limitations and Future Enhancements

12.1 Current System Limitations

12.1.1 Technical Limitations

- Windows-only: reliance on `<windows.h>` and `<conio.h>` prevents compilation on Linux or macOS without a compatibility layer.
- No unique-solution guarantee: holes are dug randomly; the puzzle may have multiple solutions.
- In-memory leaderboard cap: only 100 entries are read; older entries are silently dropped if the file grows beyond that.
- No save/load mid-game: the current puzzle state is lost if the application is closed.

12.1.2 Functional Limitations

- No undo: a mistyped digit can only be cleared, not stepped back through history.
- Single-player only: no networked or same-machine multiplayer.
- No puzzle import: puzzles cannot be loaded from a file (only entered manually in auto-solver mode).
- No difficulty adjustment after game start: the player must restart to change difficulty.

12.2 Proposed Future Enhancements

- Cross-platform port using ncurses or a lightweight portable terminal library.
- Unique-solution enforcement by running a solve-count check after each hole is dug.
- Undo/redo stack using a fixed-size move history array.
- Mid-game save using a simple text file storing the grid state and elapsed time.
- Networked leaderboard via a local TCP socket or REST API.
- Graphical front-end using raylib or SDL for a richer visual experience.

13. Learning Outcomes

- C Programming Proficiency: deep hands-on experience with pointers, 2-D arrays, structs, enums, file I/O, and the Windows Console API.
- Algorithm Design: implementing and understanding backtracking, MRV heuristic, and Forward Checking as applied to constraint-satisfaction problems.
- Modular Software Architecture: partitioning a project into compilation units with a shared header, reducing coupling and improving maintainability.
- UI/UX in a Console Context: designing a usable interface without a graphics library by leveraging terminal-width detection, cursor positioning, and ANSI colour codes.
- Problem Decomposition: breaking a complex application (generation + play + solving + leaderboard) into independently testable modules.
- Teamwork and Version Control: collaborating across source files with a shared Makefile and consistent coding conventions.
- Technical Communication: documenting design decisions, constraints, and algorithm choices in a formal project report.

Bibliography

- [1] D. Knuth, "Dancing Links," in Millenial Perspectives in Computer Science, Palgrave Macmillan, 2000. Seminal work on exact cover and Sudoku solution via Algorithm X.
- [2] S. J. Russell and P. Norvig, Artificial Intelligence: A Modern Approach, 4th ed. Pearson, 2020. Chapter 6: Constraint Satisfaction Problems; MRV heuristic and Forward Checking.
- [3] Microsoft, "Console Screen Buffer Info" Windows Dev Docs, 2024. [Online]. Available: <https://docs.microsoft.com/en-us/windows/console>
- [4] W. Harvey and M. Ginsberg, "Limited Discrepancy Search," in Proceedings of IJCAI-95, 1995. Discusses heuristic search strategies relevant to the MRV implementation.
- [5] GNU Make, Free Software Foundation, Version 4.4, 2023. [Online]. Available: <https://www.gnu.org/software/make/>. Build automation tool used for compilation.
- [6] Microsoft Corporation, "Console Reference," in Windows Developer Documentation, 2023. [Online]. Available: <https://docs.microsoft.com/en-us/windows/console/console-reference>.